SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

DryCare — Multi-Tenant Dry Cleaning & Laundry Management SaaS

Document ID: DRYCARE-SRS-001
Version: 1.0
Status: Proposed Baseline for Product Design and Development
Product Type: Multi-tenant SaaS
Platform: Responsive web application
Primary Market: India, with architecture extensible to international markets
Technology Baseline: Next.js, TypeScript, Fastify, PostgreSQL, Drizzle ORM and Redis
Audience: Product owner, business analyst, UI/UX designer, frontend developers, backend developers, QA engineers, DevOps engineers and business stakeholders

---

1. Document Purpose

This document defines the functional, technical, operational, security and quality requirements for DryCare, a commercial dry cleaning and laundry store management platform.

The document is intended to serve as the baseline for:

- Product planning and scope approval.
- UI/UX design and screen implementation.
- Database design and migration planning.
- Backend API development.
- Frontend and backend integration.
- Role-based access control.
- Quality assurance and acceptance testing.
- Production deployment and operational support.
- Future expansion into a commercially operated SaaS product.

All requirements should be converted into tracked implementation tasks. Any change to the approved scope must be recorded through change control.

1.1 Normative terminology

- **MUST:** Mandatory requirement.
- **SHOULD:** Recommended requirement that may be waived with documented justification.
- **MAY:** Optional capability.
- **Tenant:** An independent dry cleaning business using the platform.
- **Organization:** The business entity represented by a tenant.
- **Branch:** A physical store, processing facility or operational location belonging to an organization.
- **Order:** A customer transaction containing one or more garments or services.
- **Garment item:** An individually identifiable physical garment or other article accepted for service.
- **Platform administrator:** An authorized operator of the SaaS platform, distinct from a tenant's business administrator.

2. Product Overview

2.1 Product vision

DryCare will provide a centralized digital system for managing the complete garment-care lifecycle, from customer registration and garment intake through processing, inspection, billing, collection and delivery.

The system will replace disconnected registers, spreadsheets and manual tracking processes with structured records and controlled workflows.

2.2 Business objectives

The product MUST:

1. Maintain complete customer profiles and transaction histories.
2. Track individual garments through configurable processing stages.
3. Support barcode and QR-based garment identification.
4. Generate accurate service charges, invoices, receipts and payment records.
5. Track partial payments, outstanding balances, refunds and approved customer credit.
6. Manage cleaning, finishing, inspection, packing and delivery operations.
7. Provide inventory, supplier, expense and staff management.
8. Generate operational and financial reports.
9. Support multiple branches under one organization.
10. Support multiple independent organizations with strict data isolation.

11. Provide configurable SaaS subscriptions and platform administration.
12. Support reliable backups, audit logging, monitoring and recovery.

2.3 Intended users

- SaaS platform administrators.
- Business owners.
- Organization administrators.
- Branch managers.
- Reception and counter staff.
- Cleaning and finishing operators.
- Quality inspectors.
- Packing staff.
- Cashiers.
- Delivery agents.
- Accountants.
- Customers using an optional self-service portal.

2.4 Product delivery model

The product will be delivered as a responsive web application accessible through modern desktop, tablet and mobile browsers.

Dedicated native mobile applications are outside the initial scope. Customer and delivery interfaces MUST remain usable on mobile browsers.

3. Scope

3.1 Included in the product

Core operations

- Organization onboarding and tenant configuration.
- Branch and employee management.
- Authentication and role-based access control.
- Customer profiles, addresses, preferences and order history.
- Garment catalogue and service catalogue.
- Price lists, discounts, surcharges and tax configuration.
- Order intake and garment-level identification.
- Barcode and QR label generation and scanning.
- Garment condition documentation and photographs.

- Processing stages, assignments, work queues and status history.
- Quality control, rework, packing and storage locations.
- Invoicing, payments, payment allocations, refunds and customer dues.
- Pickup scheduling, dispatch, delivery and proof of handover.
- Inventory, suppliers, purchases and stock movements.
- Expenses and cashier shift reconciliation.
- Notifications and message templates.
- Operational dashboards and business reports.
- Complaints, damage claims and customer feedback.
- Audit logs and operational monitoring.
- SaaS plans, subscriptions, entitlements and tenant administration.
- Data export, backups, recovery and retention controls.

3.2 Excluded from the initial release

The following features MAY be implemented in later releases:

- Native Android and iOS applications.
- AI-based garment identification or automated damage assessment.
- Predictive demand forecasting.
- Advanced route optimization.
- Automated payroll and statutory HR management.
- Full general ledger accounting.
- Automated bank statement reconciliation.
- Franchise royalty management.
- Direct integration with every laundry machine or industrial controller.
- International tax compliance for every jurisdiction.
- Offline operation of the complete application.

Limited offline resilience, including safe recovery from interrupted requests and queued non-financial operations, SHOULD be considered during initial architecture design.

3.3 Assumptions and constraints

1. Each tenant is responsible for its own business information and operational configuration.
2. A tenant may have one or more branches.
3. Customers may be shared across branches of the same tenant.
4. Garments may be processed at a different branch or external facility when configured.
5. The platform may offer different subscription plans with feature and usage limits.
6. External payment, SMS, WhatsApp and email providers may impose separate fees and availability constraints.
7. Hardware compatibility must be verified against specific printer and scanner models.

8. Applicable Indian tax and privacy requirements must be reviewed before production.

4. System Architecture

4.1 Recommended technology stack

Layer	Technology	Responsibility
Web frontend	Next.js and TypeScript	Responsive interfaces
UI components	Tailwind CSS and shadcn/ui	Consistent visual system
Backend	Node.js, Fastify and TypeScript	Business logic and APIs
Database	PostgreSQL	Transactional source of truth
ORM	Drizzle ORM	Typed queries and schema migrations
Cache and queues	Redis with a durable job queue	Background jobs, caching and coordination
File storage	S3-compatible object storage	Garment photographs, documents and exports
Authentication	Secure server-side sessions	Login, session revocation and authorization
Payments	Configurable payment gateway	Online payments and payment verification
Notifications	Approved messaging providers	SMS, WhatsApp and email
Deployment	Docker, reverse proxy and CI/CD	Repeatable builds and controlled releases
Monitoring	Centralized logs, metrics and error tracking	Reliability and incident investigation

The implementation MUST use currently supported, mutually compatible dependency versions. Versions MUST be pinned and reviewed rather than assumed from this document.

4.2 Logical architecture

The system SHALL use a modular monolith for the initial release.

Modules SHALL have defined service boundaries, ownership of business rules and controlled access to shared data.

Logical modules:

1. Identity and access.
2. Tenant and branch management.
3. Customer management.
4. Catalogue and pricing.
5. Order management.
6. Garment tracking.
7. Production and quality control.
8. Billing and payments.
9. Pickup and delivery.
10. Inventory and purchasing.
11. Expenses and reconciliation.
12. Notifications.
13. Reporting and analytics.
14. SaaS subscriptions.
15. Platform administration.
16. Audit and compliance.

Modules SHOULD communicate through explicit service interfaces and domain events where appropriate.

4.3 Architecture principles

- PostgreSQL MUST be the authoritative source for critical business transactions.
- Redis MUST NOT be the sole store for orders, payments or audit records.
- Business rules MUST be enforced in the backend.
- Tenant ownership MUST be checked for every tenant-scoped operation.
- External integrations MUST be isolated behind provider adapters.
- Background jobs MUST be retryable and idempotent where possible.
- Financial corrections MUST preserve transaction history.
- Database schema changes MUST use reviewed, version-controlled migrations.
- All production secrets MUST be supplied through a secure secret-management mechanism.

5. Multi-Tenant SaaS Requirements

5.1 Tenant isolation

The system MUST isolate all business data belonging to different tenants.

Tenant-owned records MUST carry an organization identifier directly or be linked through a verified tenant-owned parent record.

The initial architecture SHOULD use a shared PostgreSQL database with tenant-scoped records, composite foreign-key constraints where appropriate and explicit tenant authorization.

PostgreSQL row-level security MAY be added as defense in depth. If used, application connection handling MUST prevent unsafe reuse of tenant context across pooled connections.

5.2 Tenant provisioning

The platform MUST support:

- Creating an organization.
- Registering the initial business owner.
- Selecting a subscription plan or trial.
- Creating the primary branch.
- Configuring business details and regional settings.
- Setting the initial service catalogue and prices.
- Inviting staff.
- Configuring invoice numbering and notification providers.
- Activating or suspending an organization under authorized platform controls.

5.3 Subscription entitlements

The platform MUST support configurable plan definitions and feature entitlements.

Potential entitlements include:

- Number of branches.
- Number of staff accounts.
- Monthly order volume.
- Customer portal access.
- Advanced reports.
- Inventory management.
- Delivery management.
- API integrations.
- Data export.
- Support tier.

Entitlements MUST be enforced on the backend. Frontend visibility alone MUST NOT be treated as authorization.

5.4 Tenant lifecycle

The system MUST support active, trial, past-due, suspended and terminated tenant states.

Suspension MUST NOT automatically delete business data.

The system MUST define policies for:

- Subscription cancellation.
- Payment failure and grace periods.
- Tenant reactivation.
- Data export.
- Data retention.
- Deletion requests.
- Backup retention.
- Final account termination.

6. Functional Requirements

6.1 Identity and authentication

FR-AUTH-001: Users MUST be able to authenticate using their configured credentials.

FR-AUTH-002: Passwords MUST be stored using a modern, appropriately configured password hashing algorithm.

FR-AUTH-003: The system MUST support logout and server-side session revocation.

FR-AUTH-004: Password reset MUST use time-limited, single-use verification tokens or equivalent secure verification.

FR-AUTH-005: The system SHOULD support multi-factor authentication for platform administrators and tenant owners.

FR-AUTH-006: Repeated failed authentication attempts MUST be rate-limited and monitored.

FR-AUTH-007: Suspended users MUST NOT access protected business operations.

FR-AUTH-008: User invitations MUST expire and MUST be accepted through a verified workflow.

6.2 Tenant and branch management

FR-TEN-001: Platform administrators MUST be able to provision and manage tenant accounts.

FR-TEN-002: Tenant administrators MUST be able to update permitted business settings.

FR-TEN-003: Organizations MUST be able to create and manage branches within their plan limits.

FR-TEN-004: Branch records MUST include a name, address, timezone, contact details and operating configuration.

FR-TEN-005: Branch transfers MUST record the origin, destination, garments, actor, timestamp and transfer status.

FR-TEN-006: Organization-wide and branch-specific reporting MUST respect the requesting user's permissions.

6.3 Customer management

FR-CRM-001: Staff MUST be able to create and update customer profiles.

FR-CRM-002: Customer profiles MUST support name, phone number, alternate contact, email, addresses and relevant service preferences.

FR-CRM-003: Customer search MUST support customer ID, phone number, name and order number.

FR-CRM-004: The system MUST detect potential duplicate customers.

FR-CRM-005: Authorized users MUST be able to view the customer's orders, invoices, payments, complaints and outstanding balance.

FR-CRM-006: Customers MAY have multiple pickup and delivery addresses.

FR-CRM-007: Marketing consent and communication preferences MUST be stored separately from operational notification preferences.

FR-CRM-008: Customer records MUST NOT be hard-deleted if doing so would break legally required financial or audit history.

FR-CRM-009: Customer merges MUST preserve relevant order references and an audit record.

6.4 Catalogue and pricing

FR-CAT-001: Administrators MUST be able to configure garment types and service types.

FR-CAT-002: Services MUST support configurable units, such as per garment, per kilogram, per pair or per service package.

FR-CAT-003: Pricing MUST support branch-specific and effective-dated price lists.

FR-CAT-004: Pricing MUST support additional charges, express surcharges, discounts and applicable taxes.

FR-CAT-005: Historical order prices MUST remain unchanged when future prices are edited.

FR-CAT-006: Price overrides MUST require appropriate permission and record the reason.

FR-CAT-007: The system MUST validate the price, currency, quantity and tax configuration before finalizing a bill.

6.5 Order management

FR-ORD-001: Staff MUST be able to create an order for an existing or new customer.

FR-ORD-002: An order MUST contain one or more order items unless explicitly configured for a valid service-only transaction.

FR-ORD-003: Each garment MUST support its own service, quantity, price, condition and instructions.

FR-ORD-004: The system MUST generate unique order identifiers within the defined numbering scope.

FR-ORD-005: Orders MUST store received time, promised completion time, branch and order source.

FR-ORD-006: Orders MUST support advance payments, partial payments and outstanding balances.

FR-ORD-007: Order changes MUST be audited.

FR-ORD-008: Cancellation MUST follow configured permission and financial rules.

FR-ORD-009: Partial completion MUST be supported.

FR-ORD-010: Reprinting a label or invoice MUST NOT create duplicate financial transactions.

6.6 Garment intake and identification

FR-GAR-001: Each physical garment MUST receive a unique identifier.

FR-GAR-002: The system MUST generate a scannable barcode or QR code for each garment.

FR-GAR-003: The garment record SHOULD support garment type, colour, fabric, brand, condition, service, care instructions and photographs.

FR-GAR-004: Staff MUST be able to record pre-existing stains, tears, missing buttons and other visible damage.

FR-GAR-005: Each garment MUST be linked to its current order and processing history.

FR-GAR-006: The system MUST support tag reprinting and tag replacement without losing historical identity.

FR-GAR-007: The system MUST detect invalid or duplicated tag assignments.

6.7 Production and workflow

FR-WF-001: Processing workflows MUST be configurable by service type.

FR-WF-002: The system MUST support stages such as received, inspected, sorted, cleaning, finishing, quality control, packing, ready and delivered.

FR-WF-003: Every stage transition MUST record actor, time, previous stage and new stage.

FR-WF-004: Stage transitions MUST validate prerequisites.

FR-WF-005: The system MUST support employee assignment and work queues.

FR-WF-006: The system MUST identify overdue orders based on promised completion time.

FR-WF-007: The system MUST support rework and quality rejection.

FR-WF-008: The system SHOULD support physical rack, shelf or storage-location assignment.

FR-WF-009: Orders MUST NOT be considered fully delivered while required garments remain undelivered, unless an authorized partial-delivery workflow is used.

6.8 Quality control and claims

FR-QC-001: Configurable inspection checklists MUST be supported.

FR-QC-002: Inspection results MUST identify the garment, inspector, timestamp and outcome.

FR-QC-003: Failed inspections MUST trigger rework or an authorized exception workflow.

FR-QC-004: Damage claims MUST capture relevant evidence, investigation notes, decision and resolution.

FR-QC-005: Compensation, refunds and write-offs MUST require appropriate authorization.

6.9 Billing and payments

FR-PAY-001: The system MUST generate itemized invoices and receipts.

FR-PAY-002: The system MUST support cash, UPI, card, bank transfer and supported payment gateways.

FR-PAY-003: The system MUST support advances and partial payments.

FR-PAY-004: Payments MUST be recorded independently of order status.

FR-PAY-005: Payment allocation MUST determine how much is applied to each invoice or order.

FR-PAY-006: Refunds MUST reference the original payment and follow an authorized workflow.

FR-PAY-007: External payment webhooks MUST be authenticated and processed idempotently.

FR-PAY-008: The system MUST reconcile gateway transactions against internal records.

FR-PAY-009: Outstanding balances MUST be calculated from valid charges, adjustments, payments and refunds according to documented accounting rules.

FR-PAY-010: Financial records MUST NOT be silently overwritten.

FR-PAY-011: The system MUST support configurable invoice sequences and appropriate tax fields.

FR-PAY-012: Cashier shift opening, collections, adjustments and shift closing MUST be recorded.

6.10 Pickup and delivery

FR-DEL-001: Authorized users MUST be able to schedule pickups and deliveries.

FR-DEL-002: Delivery records MUST support customer address, time slot, assigned driver and instructions.

FR-DEL-003: The system MUST support dispatch manifests and garment verification.

FR-DEL-004: Delivery completion MUST record the relevant timestamp and confirmation method.

FR-DEL-005: Failed delivery attempts MUST support reasons and rescheduling.

FR-DEL-006: Drivers MUST only access assigned delivery records and necessary customer details.

6.11 Inventory and purchasing

FR-INV-001: The system MUST support inventory items, units, suppliers and stock movements.

FR-INV-002: Stock receipts, consumption, returns, wastage and adjustments MUST be recorded.

FR-INV-003: Low-stock thresholds MUST be configurable.

FR-INV-004: Purchase orders and purchase records SHOULD be supported.

FR-INV-005: Inventory changes MUST record the actor, time and reason.

FR-INV-006: Automated consumption estimates MAY be configured by service type.

6.12 Expenses and reconciliation

FR-FIN-001: Authorized staff MUST be able to record business expenses.

FR-FIN-002: Expenses MUST support category, branch, date, amount, payment method and supporting documentation.

FR-FIN-003: The system MUST distinguish billed sales, net revenue under the configured accounting policy, collections, outstanding receivables and expenses.

FR-FIN-004: Financial reports MUST reconcile with underlying transaction records.

FR-FIN-005: Corrections to financial records MUST preserve an audit trail.

6.13 Notifications

FR-NOT-001: The system MUST support configurable notification templates.

FR-NOT-002: Supported channels SHOULD include SMS, WhatsApp Business Platform and email.

FR-NOT-003: Eligible events SHOULD include booking, pickup scheduling, ready status, delivery dispatch, payment receipt, delay and feedback requests.

FR-NOT-004: Notification attempts MUST record provider response and delivery status when available.

FR-NOT-005: Retry policies MUST prevent uncontrolled duplicate messages.

FR-NOT-006: Marketing communications MUST respect consent and opt-out preferences.

6.14 Reports and analytics

FR-REP-001: The dashboard MUST display authorized operational and financial KPIs.

FR-REP-002: Reports MUST support date-range and relevant branch filters.

FR-REP-003: Reports SHOULD support service, employee, payment mode and status filters where appropriate.

FR-REP-004: The system MUST provide sales, order, garment, payment, dues, inventory, expense and delivery reports.

FR-REP-005: Exports MUST enforce the same authorization and tenant isolation as on-screen reports.

FR-REP-006: Report definitions MUST document how taxes, refunds, cancelled orders and partial payments are treated.

6.15 SaaS subscriptions and platform administration

FR-SAAS-001: Platform administrators MUST be able to create and update subscription plans.

FR-SAAS-002: Plans MUST support pricing, billing interval, feature entitlements and usage limits.

FR-SAAS-003: Subscription changes MUST be recorded with effective dates and audit history.

FR-SAAS-004: The system MUST support trial, active, past-due, suspended and cancelled subscription states.

FR-SAAS-005: Failed subscription payments MUST follow configurable retry and grace-period rules.

FR-SAAS-006: Tenant suspension MUST NOT silently delete data.

FR-SAAS-007: Platform support access to tenant data MUST be restricted, authorized and audited.

6.16 Audit and compliance

FR-AUD-001: Sensitive actions MUST generate audit events.

FR-AUD-002: Audit events SHOULD include tenant, actor, action, resource, timestamp, request correlation ID and relevant before/after values.

FR-AUD-003: Audit records MUST be protected against unauthorized modification.

FR-AUD-004: Logs MUST NOT contain passwords, authentication tokens, payment secrets or unnecessary sensitive customer information.

FR-AUD-005: The system MUST support configurable data retention and export policies.

7. Screen-by-Screen UI Specification

7.1 Global application shell

Requirements

- Responsive sidebar or navigation drawer.
- Organization and branch selector.
- Global search for customers, orders and garment tags.
- User menu, profile, security settings and logout.
- Notifications and alerts.
- Breadcrumbs and page titles.
- Permission-aware navigation.
- Loading, empty, error and success states.
- Accessible forms, keyboard navigation and clear validation messages.

The active tenant and branch context MUST be visible wherever they affect the displayed data.

7.2 Public and authentication screens

Screen A01 — Landing and sign-in

Components:

- Product branding.

- Email or username and password.
- Password visibility toggle.
- Forgot-password link.
- Secure login submission.
- Error messages and rate-limit feedback.

Acceptance:

- Invalid credentials MUST NOT reveal whether an account exists unless explicitly allowed by a secure design.
- Successful login MUST establish the correct authorized session.

Screen A02 — Password reset

Components:

- Account identifier input.
- Verification instructions.
- New password and confirmation.
- Success and failure states.

Acceptance:

- Reset tokens MUST expire and be single-use.

Screen A03 — Invitation acceptance

Components:

- Invited organization name.
- Invited email address.
- Password setup or supported identity-provider flow.
- Terms and privacy links.

Acceptance:

- Expired, revoked or previously used invitations MUST be rejected.

7.3 Organization and onboarding screens

Screen B01 — Business onboarding wizard

Steps:

1. Business details.

2. Primary branch details.
3. Regional settings and currency.
4. Services and garment types.
5. Initial prices and taxes.
6. Invoice configuration.
7. Staff invitations.
8. Messaging and payment integrations.
9. Review and activate.

Acceptance:

- Required configuration MUST be validated before activation.
- Incomplete onboarding MUST be resumable.

Screen B02 — Branch management

Components:

- Branch table.
- Search and filters.
- Create/edit branch dialog.
- Address and contact details.
- Timezone and operational settings.
- Branch activation state.

Screen B03 — Organization settings

Sections:

- Business identity.
- Addresses and contact details.
- Currency and regional settings.
- Invoice numbering.
- Tax configuration.
- Notifications.
- Data retention and privacy.
- Security and session policy.

7.4 Owner and manager screens

Screen C01 — Main dashboard

Cards and visualizations:

- Gross billed sales.
- Net sales or revenue under the configured definition.
- Payments collected.
- Outstanding receivables.
- Orders received and completed.
- Garments awaiting processing.
- Orders ready for collection.
- Overdue orders.
- Rework rate.
- Branch comparison.

Every metric MUST use a documented definition and a consistent date range.

Screen C02 — Order management list

Columns:

- Order number.
- Customer.
- Branch.
- Received date.
- Promised completion.
- Garment count.
- Total.
- Amount paid.
- Outstanding balance.
- Order status.
- Payment status.

Actions:

- Search.
- Filter.
- Open order.
- Create order.
- Print authorized documents.
- Export permitted results.

Screen C03 — Production board

Views:

- Received.
- Inspection.
- Sorting.

- Cleaning.
- Finishing.
- Quality control.
- Rework.
- Packing.
- Ready.

Features:

- Stage filters.
- Due-date indicators.
- Employee assignments.
- Garment counts.
- Scan-to-transition.
- Authorized exception handling.

Screen C04 — Reports

Features:

- Date-range filters.
- Branch filters.
- Service and employee filters.
- KPI summaries.
- Tabular details.
- Pagination.
- Export.
- Clear report definitions.

7.5 Customer screens

Screen D01 — Customer directory

Columns:

- Customer ID.
- Name.
- Phone.
- Customer type.
- Last order date.
- Total orders.
- Outstanding balance.

Actions:

- Search.
- Create customer.
- Open profile.
- Edit authorized fields.
- Merge duplicates with approval.
- Export permitted data.

Screen D02 — Customer creation/edit

Fields:

- Full name.
- Primary and alternate phone.
- Email.
- Customer category.
- Address list.
- Preferred language.
- Operational communication preferences.
- Marketing consent.
- Relevant garment-care instructions.

Screen D03 — Customer profile

Tabs:

1. Overview.
2. Addresses.
3. Orders.
4. Garments.
5. Invoices and payments.
6. Outstanding balance.
7. Complaints and claims.
8. Preferences and consent.
9. Authorized notes.

The profile MUST provide a clear link to create an order for the selected customer.

7.6 POS and garment intake screens

Screen E01 — New order

Layout:

- Customer search and registration.

- Branch and order source.
- Garment/service selector.
- Item list.
- Promised completion date.
- Intake notes.
- Pricing summary.
- Advance payment.
- Save and print actions.

Acceptance:

- Saving an order MUST create a consistent order and garment record set.
- Repeated submissions MUST NOT create duplicates.
- Price calculations MUST be performed or validated by the backend.

Screen E02 — Garment detail editor

Fields:

- Garment type.
- Quantity or individual item count.
- Colour.
- Fabric and brand, where known.
- Service type.
- Price.
- Condition checklist.
- Stain and damage notes.
- Photographs.
- Care instructions.
- Barcode or QR identifier.

Screen E03 — Barcode and label printing

Features:

- Label preview.
- Configured label dimensions.
- Order and garment identifiers.
- Barcode or QR code.
- Reprint option.
- Printer status and error feedback.

The system MUST verify that a label is linked to the correct garment before printing.

Screen E04 — Order confirmation and receipt

Displays:

- Order number.
- Customer details.
- Garment list.
- Services and charges.
- Applicable tax details.
- Discounts.
- Advance payment.
- Remaining balance.
- Promised completion.
- Collection or delivery instructions.

7.7 Production screens

Screen F01 — Garment scan station

Features:

- Scanner input.
- Garment identification.
- Current stage.
- Next permitted stage.
- Processing instructions.
- Employee assignment.
- Success or failure feedback.

Screen F02 — Quality inspection

Features:

- Garment details.
- Intake condition photographs.
- Inspection checklist.
- Pass, fail or authorized exception.
- Defect notes.
- Rework destination.
- Inspector identity and timestamp.

Screen F03 — Packing and storage

Features:

- Scan garment.

- Group garments by order.
- Verify expected garment count.
- Assign storage location.
- Mark ready for collection.
- Record missing-item exceptions.

7.8 Billing screens

Screen G01 — Invoice detail

Displays:

- Invoice number.
- Customer and order reference.
- Line items.
- Tax breakdown.
- Discounts.
- Total amount.
- Payments and allocations.
- Outstanding amount.
- Invoice history.

Screen G02 — Payment collection

Features:

- Amount due.
- Payment amount.
- Payment method.
- Transaction reference.
- Split payment support.
- Cashier identity.
- Receipt generation.

Screen G03 — Refund and adjustment

Features:

- Original transaction.
- Refund amount.
- Reason.
- Supporting evidence where needed.
- Approval workflow.
- Gateway status.

- Audit history.

7.9 Inventory and expense screens

Screen H01 — Inventory dashboard

Displays:

- Current stock.
- Low-stock items.
- Recent movements.
- Pending purchases.
- Stock valuation where configured.

Screen H02 — Inventory item and supplier

Fields:

- SKU.
- Name and category.
- Unit.
- Supplier.
- Minimum stock.
- Purchase cost.
- Stock movements.
- Relevant safety and batch details.

Screen H03 — Purchase and stock receipt

Features:

- Purchase order.
- Supplier.
- Item quantities.
- Received quantities.
- Invoice reference.
- Stock update.
- Discrepancy handling.

Screen H04 — Expense management

Features:

- Expense category.

- Amount and currency.
- Branch.
- Date.
- Payment method.
- Attachment.
- Approval status.
- Reporting category.

7.10 Delivery screens

Screen I01 — Pickup and delivery scheduler

Fields:

- Customer.
- Address.
- Date and time slot.
- Service request.
- Assigned driver.
- Notes.
- Status.

Screen I02 — Driver worklist

Features:

- Assigned stops.
- Necessary contact details.
- Address and instructions.
- Order or bag reference.
- Pickup/delivery action.
- Failed attempt reason.
- Confirmation capture.

Screen I03 — Dispatch and proof of delivery

Features:

- Dispatch manifest.
- Garment count verification.
- Driver assignment.
- OTP or signature confirmation.
- Timestamp.
- Optional authorized photograph.

- Delivery outcome.

7.11 Administration screens

Screen J01 — Users and permissions

Features:

- User list.
- Role assignment.
- Branch assignment.
- Invitations.
- Active/suspended state.
- Session revocation.
- Permission matrix.

Screen J02 — Service and price configuration

Features:

- Service catalogue.
- Garment types.
- Units.
- Price lists.
- Effective dates.
- Branch overrides.
- Discount rules.
- Tax settings.

Screen J03 — Notification templates

Features:

- Channel selection.
- Event trigger.
- Template variables.
- Language.
- Enable/disable.
- Preview.
- Test send.
- Delivery logs.

Screen J04 — SaaS platform administration

Features:

- Tenant directory.
- Subscription state.
- Plan and entitlements.
- Branch and user usage.
- Payment status.
- Support notes.
- Suspension/reactivation.
- Audited support access.

Screen J05 — Audit and system health

Features:

- Searchable audit records.
- User and tenant filters.
- Time range.
- Event category.
- Failed jobs.
- Integration errors.
- Background queue health.
- Operational alerts.

7.12 Customer self-service portal

The portal is an optional phased feature.

Screens:

- Sign-in and verification.
- Customer profile.
- Order list.
- Order details and status.
- Invoices and payment history.
- Pickup booking.
- Online payment.
- Feedback and complaints.
- Communication preferences.

Customers MUST only be able to access their own authorized records.

8. Core Business Workflows

8.1 New order workflow

1. Search for the customer.
2. Create a customer if required.
3. Select the receiving branch.
4. Add garment items.
5. Record garment condition and instructions.
6. Select service and applicable price.
7. Calculate charges and taxes.
8. Set promised completion time.
9. Record advance payment if applicable.
10. Create the order and associated garments transactionally.
11. Generate identifiers and print labels.
12. Issue the order receipt.
13. Schedule eligible notifications.

Failure at a critical transaction step MUST NOT leave an order partially created without a recoverable state.

8.2 Garment processing workflow

1. Scan the garment.
2. Verify its identity and current stage.
3. Confirm prerequisites.
4. Record the new stage and employee.
5. Save timestamp and relevant notes.
6. Record exceptions or inspection results.
7. Update the order's derived operational status.
8. Notify eligible users or customers when appropriate.

8.3 Quality-control workflow

1. Scan garment.
2. Load intake condition and care instructions.
3. Complete configured inspection checklist.
4. Record pass or fail.
5. If failed, create a rework record.
6. If passed, allow packing.
7. Preserve inspection history.

8.4 Payment workflow

1. Load the applicable invoice or order balance.
2. Validate the payment amount and method.
3. Initiate the payment process if using a gateway.
4. Verify payment status using the trusted gateway response or authenticated webhook.
5. Record the transaction idempotently.
6. Allocate the payment.
7. Recalculate the balance.
8. Generate a receipt.
9. Update payment reports.

A browser redirect MUST NOT be treated as sufficient proof of payment.

8.5 Delivery workflow

1. Verify garment readiness and applicable quality approval.
2. Verify the outstanding balance and configured credit rules.
3. Create dispatch manifest.
4. Assign driver.
5. Verify garments before handover.
6. Record dispatch.
7. Capture delivery outcome.
8. Record final collection or authorized credit.
9. Close the order only when closure conditions are satisfied.

8.6 Subscription workflow

1. Tenant selects a plan.
2. System calculates applicable charges.
3. Payment is initiated where required.
4. Subscription status is updated from a verified payment result.
5. Entitlements are applied.
6. Renewal and failure events are processed idempotently.
7. Grace-period and suspension policies are enforced.
8. Tenant receives appropriate billing notifications.

9. Order, Garment and Payment State Models

9.1 Order state

Suggested operational states:

- Draft.
- Received.
- In processing.
- Partially ready.
- Ready for collection.
- Partially delivered.
- Delivered.
- Cancelled.

The implementation MAY derive the order's visible status from garment states, but the derivation rules MUST be documented and consistent.

9.2 Garment state

Suggested states:

- Received.
- Inspection.
- Sorting.
- Cleaning.
- Drying.
- Finishing.
- Quality control.
- Rework.
- Packing.
- Ready.
- Dispatched.
- Delivered.
- Exception.
- Cancelled or rejected.

Not every service uses every stage. Workflow configuration MUST define permitted transitions and prerequisites.

9.3 Payment state

Suggested payment states:

- Pending.
- Processing.
- Succeeded.
- Failed.

- Cancelled.
- Partially refunded.
- Refunded.

Invoice payment state MUST be calculated independently as unpaid, partially paid, paid, overdue or otherwise configured.

9.4 Transition controls

- Invalid transitions MUST be rejected.
- Required inspection MUST precede normal release.
- Delivered garments MUST NOT silently return to an earlier stage.
- Corrections MUST use authorized exception or reversal workflows.
- Concurrent updates MUST not silently overwrite one another.
- Every material transition MUST be auditable.

10. Database Schema Specification

10.1 Common conventions

The database SHOULD follow these conventions:

- UUID primary keys for internal entities.
- `timestamptz` for event timestamps.
- Exact decimal or integer minor-unit storage for monetary values.
- Explicit currency codes for monetary amounts.
- Foreign keys for referential integrity.
- Appropriate unique constraints and indexes.
- `created_at` and `updated_at` on mutable business records.
- `organization_id` on tenant-owned records, directly or through an enforced ownership relationship.
- Soft deletion only where appropriate.
- Immutable or append-only records for audit and financial events.

All schema examples are logical specifications. Exact SQL types, enums and migrations must be finalized during implementation.

10.2 Identity and tenant tables

`organizations`

Fields:

- `id`
- `name`
- `legal_name`
- `slug`
- `status`
- `default_currency`
- `default_timezone`
- `settings_json`
- `created_at`
- `updated_at`

Constraints:

- Unique organization slug.
- Valid organization state.
- Valid currency and timezone.

`branches`

Fields:

- `id`
- `organization_id`
- `name`
- `branch_code`
- `address_json`
- `phone`
- `timezone`
- `status`
- `created_at`
- `updated_at`

Constraints:

- Unique branch code within organization.
- Branch ownership MUST be enforced.

`users`

Fields:

- `id`
- `email`
- `phone`
- `password_hash` or identity-provider reference
- `name`
- `status`
- `last_login_at`
- `created_at`
- `updated_at`

Global or tenant-scoped identity rules MUST be chosen consistently. A user identity MUST NOT be used to bypass organization authorization.

`organization_memberships`

Fields:

- `id`
- `organization_id`
- `user_id`
- `status`
- `created_at`

Purpose:

- Links users to tenant organizations.

`roles`, `permissions`, `role_permissions`, `user_roles`

Purpose:

- Define roles, permissions and user assignments.

`user_branch_assignments`

Purpose:

- Restricts a user to permitted branches.

10.3 Customer tables

`customers`

Fields:

- id
- organization_id
- customer_code
- name
- phone
- alternate_phone
- email
- customer_type
- preferred_language
- operational_preferences_json
- marketing_consent
- status
- created_at
- updated_at

customer_addresses

Fields:

- id
- organization_id
- customer_id
- label
- address_json
- is_default
- created_at
- updated_at

customer_preferences

Fields:

- id
- organization_id
- customer_id
- preference_type
- value_json
- updated_at

10.4 Catalogue and pricing tables

`garment_types`

Fields:

- `id`
- `organization_id`
- `name`
- `description`
- `default_unit`
- `active`

`services`

Fields:

- `id`
- `organization_id`
- `name`
- `service_code`
- `description`
- `unit_type`
- `workflow_template_id`
- `active`

`price_lists`

Fields:

- `id`
- `organization_id`
- `branch_id`, nullable where organization-wide pricing applies
- `name`
- `currency`
- `effective_from`
- `effective_to`
- `status`

`service_prices`

Fields:

- `id`
- `organization_id`
- `price_list_id`
- `garment_type_id`
- `service_id`
- `unit_price`
- `tax_configuration_id`
- `minimum_charge`
- `effective_from`
- `effective_to`

Price history MUST be preserved. Changes MUST NOT rewrite historical order item prices.

10.5 Order and garment tables

`orders`

Fields:

- `id`
- `organization_id`
- `branch_id`
- `customer_id`
- `order_number`
- `order_source`
- `operational_status`
- `received_at`
- `promised_at`
- `closed_at`
- `subtotal`
- `discount_total`
- `tax_total`
- `grand_total`
- `currency`
- `notes`
- `created_by`
- `created_at`
- `updated_at`

Constraints:

- Unique order number within the defined tenant/branch numbering scope.
- Valid customer and branch ownership.
- Totals consistent with the documented pricing rules.

`order_items`

Fields:

- `id`
- `organization_id`
- `order_id`
- `garment_type_id`
- `service_id`
- `quantity`
- `unit_price`
- `discount_amount`
- `tax_amount`
- `line_total`
- `fabric`
- `colour`
- `brand`
- `condition_notes`
- `care_instructions`
- `status`
- `created_at`
- `updated_at`

Where several physical garments are included, the implementation MUST maintain individual garment records.

`garments`

Fields:

- `id`
- `organization_id`
- `order_id`
- `order_item_id`
- `tag_code`

- garment_type_id
- service_id
- current_status
- current_branch_id
- storage_location_id
- condition_data_json
- created_at
- updated_at

Constraints:

- Unique tag code within the required uniqueness scope.
- Valid ownership relationships.
- No duplicate active assignment of a physical garment identity.

garment_photos

Fields:

- id
- organization_id
- garment_id
- storage_key
- photo_type
- created_by
- created_at

garment_status_history

Fields:

- id
- organization_id
- garment_id
- previous_status
- new_status
- actor_user_id
- location_id
- notes
- occurred_at
- correlation_id

`workflow_templates`, `workflow_stages`, `workflow_transitions`

Purpose:

- Define service-specific workflows and allowed state transitions.

`quality_inspections`

Fields:

- `id`
- `organization_id`
- `garment_id`
- `inspector_user_id`
- `outcome`
- `checklist_json`
- `notes`
- `created_at`

`rework_records`

Purpose:

- Tracks failed inspections, rework reasons, reassignment and resolution.

`storage_locations`

Purpose:

- Defines branch, rack, shelf and storage positions.

10.6 Billing and financial tables

`invoices`

Fields:

- `id`
- `organization_id`
- `branch_id`
- `customer_id`
- `order_id`

- invoice_number
- status
- currency
- subtotal
- discount_total
- tax_total
- grand_total
- issued_at
- due_at
- created_at

invoice_items

Fields:

- id
- organization_id
- invoice_id
- description_snapshot
- quantity
- unit_price
- discount_amount
- tax_rate_snapshot
- tax_amount
- line_total

payments

Fields:

- id
- organization_id
- branch_id
- customer_id
- payment_reference
- payment_method
- amount
- currency
- status
- provider_reference

- received_at
- created_by
- created_at

payment_allocations

Fields:

- id
- organization_id
- payment_id
- invoice_id
- allocated_amount
- created_at

refunds

Fields:

- id
- organization_id
- payment_id
- amount
- reason
- status
- approved_by
- provider_reference
- created_at

credit_notes, cashier_shifts, cashier_shift_movements

Purpose:

- Track adjustments, shift opening and closing, cash movements and reconciliation.

All financial operations MUST use database transactions and idempotency controls where appropriate.

10.7 Inventory and expense tables

Tables:

- `inventory_items`
- `suppliers`
- `purchase_orders`
- `purchase_order_items`
- `stock_movements`
- `inventory_adjustments`
- `expense_categories`
- `expenses`
- `expense_attachments`

Stock movements SHOULD be append-only. Current stock SHOULD be derived from movements or maintained through a reconciled transactional balance mechanism.

10.8 Delivery and customer service tables

Tables:

- `pickup_deliveries`
- `delivery_items`
- `delivery_events`
- `delivery_assignments`
- `complaints`
- `damage_claims`
- `customer_feedback`

Delivery events MUST record timestamps and outcomes. Proof-of-delivery files MUST use controlled access.

10.9 Notification and audit tables

Tables:

- `notification_templates`
- `notification_jobs`
- `notification_attempts`
- `audit_logs`
- `integration_events`
- `idempotency_records`
- `outbox_events`

Purpose:

- Track delivery attempts.
- Support reliable background processing.
- Record integration results.
- Prevent duplicate processing.
- Maintain auditability.

10.10 SaaS subscription tables

Tables:

- `subscription_plans`
- `plan_entitlements`
- `subscriptions`
- `subscription_items`
- `subscription_invoices`
- `subscription_payments`
- `subscription_events`
- `usage_counters`

These tables MUST distinguish platform billing from a tenant's own customer invoices.

10.11 Indexing and constraints

Indexes SHOULD cover:

- Organization and branch ownership.
- Customer phone and customer code.
- Order number and promised completion.
- Garment tag code.
- Current garment and order status.
- Invoice number and invoice status.
- Payment provider reference.
- Event timestamps.
- Notification retry status.
- Subscription state and renewal date.

Search behavior for phone numbers MUST be normalized consistently. Unique constraints MUST be chosen according to the business rules, including whether duplicate phone numbers are permitted.

11. REST API Specification

11.1 API conventions

Base path: `/api/v1`

All endpoints MUST:

- Authenticate the caller where required.
- Validate request bodies, query parameters and path parameters.
- Enforce organization and branch authorization.
- Return consistent error responses.
- Use pagination for collection endpoints.
- Use documented date and monetary formats.
- Apply rate limits to sensitive operations.
- Log a correlation ID.
- Avoid exposing internal stack traces.

Suggested error response:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request contains invalid fields.",
    "details": [],
    "requestId": "request-correlation-id"
  }
}
```

11.2 Authentication endpoints

Method	Endpoint	Purpose
POST	`/auth/login`	Authenticate
POST	`/auth/logout`	Revoke session
GET	`/auth/session`	Return current session and authorized context
POST	`/auth/forgot-password`	Initiate password reset

POST	`/auth/reset-password`	Complete password reset
POST	`/auth/accept-invitation`	Accept staff invitation
POST	`/auth/mfa/verify`	Verify configured MFA challenge

11.3 Tenant and branch endpoints

Method	Endpoint	Purpose
POST	`/organizations`	Provision organization
GET	`/organizations/current`	Get current organization
PATCH	`/organizations/current`	Update authorized settings
GET	`/branches`	List accessible branches
POST	`/branches`	Create branch
GET	`/branches/:branchId`	Get branch
PATCH	`/branches/:branchId`	Update branch
POST	`/branches/:branchId/transfers`	Initiate transfer workflow

11.4 Customer endpoints

Method	Endpoint	Purpose
GET	`/customers`	Search and paginate
POST	`/customers`	Create customer
GET	`/customers/:customerId`	Get profile
PATCH	`/customers/:customerId`	Update profile
GET	`/customers/:customerId/orders`	Get order history

GET	`/customers/:customerId/invoices`	Get invoices
GET	`/customers/:customerId/payments`	Get payment history
GET	`/customers/:customerId/balance`	Get authorized balance summary
POST	`/customers/:customerId/addresses`	Add address
PATCH	`/customers/:customerId/addresses/:addressId`	Update address
POST	`/customers/:customerId/merge`	Execute authorized merge

Customer search MUST not reveal information outside the caller's tenant.

11.5 Catalogue and pricing endpoints

Method	Endpoint	Purpose
GET	`/garment-types`	List garment types
POST	`/garment-types`	Create garment type
PATCH	`/garment-types/:id`	Update garment type
GET	`/services`	List services
POST	`/services`	Create service
PATCH	`/services/:id`	Update service
GET	`/price-lists`	List price lists
POST	`/price-lists`	Create price list
POST	`/price-lists/:id/prices`	Add price entries
PATCH	`/service-prices/:id`	Update future-effective price

11.6 Order and garment endpoints

Method	Endpoint	Purpose
GET	`/orders`	Search orders
POST	`/orders`	Create order
GET	`/orders/:orderId`	Get order details
PATCH	`/orders/:orderId`	Update permitted order fields
POST	`/orders/:orderId/cancel`	Cancel order
POST	`/orders/:orderId/partial-delivery`	Record authorized partial delivery
GET	`/garments/:garmentId`	Get garment details
GET	`/garments/by-tag/:tagCode`	Find garment by tag
POST	`/garments/:garmentId/transitions`	Request status transition
POST	`/garments/:garmentId/photos`	Register authorized photo upload
POST	`/orders/:orderId/labels/reprint`	Reprint permitted labels

The order creation endpoint SHOULD accept a validated request containing customer, branch, garment items, services, due date and optional initial payment intent. It MUST calculate or validate prices on the server.

11.7 Production and quality endpoints

Method	Endpoint	Purpose
GET	`/production/queues`	Get authorized work queues
POST	`/garments/:id/assignments`	Assign operator
POST	`/garments/:id/inspections`	Submit inspection

POST	`/garments/:id/rework`	Create rework record
POST	`/garments/:id/storage-location`	Assign location
GET	`/production/overdue`	Get overdue work

11.8 Invoice and payment endpoints

Method	Endpoint	Purpose
GET	`/invoices`	Search invoices
POST	`/invoices`	Create invoice where permitted
GET	`/invoices/:invoiceId`	Get invoice
POST	`/invoices/:invoiceId/payments`	Record permitted collection
GET	`/payments/:paymentId`	Get payment details
POST	`/payments/:paymentId/refunds`	Initiate refund
GET	`/payments/reconciliation`	Authorized reconciliation report
POST	`/cashier-shifts`	Open shift
POST	`/cashier-shifts/:id/close`	Close shift

Payment endpoints MUST validate authorization, amount, currency, idempotency and current transaction state.

11.9 Delivery and inventory endpoints

Method	Endpoint	Purpose
GET	`/deliveries`	Search deliveries
POST	`/deliveries`	Schedule pickup/delivery

POST	`/deliveries/:id/assign`	Assign driver
POST	`/deliveries/:id/dispatch`	Dispatch order
POST	`/deliveries/:id/complete`	Record delivery outcome
GET	`/inventory/items`	List inventory
POST	`/inventory/items`	Create item
GET	`/inventory/movements`	View stock movements
POST	`/inventory/movements`	Record authorized movement
POST	`/purchase-orders`	Create purchase order
POST	`/expenses`	Record expense

11.10 Notifications, reporting and subscriptions

Method	Endpoint	Purpose
GET	`/notifications`	View authorized notification records
POST	`/notifications/test`	Test configured channel
GET	`/reports/sales`	Sales report
GET	`/reports/orders`	Order report
GET	`/reports/garments`	Garment report
GET	`/reports/receivables`	Outstanding dues
GET	`/reports/inventory`	Inventory report
GET	`/reports/staff-performance`	Staff report

GET	`/subscription-plans`	List available plans
POST	`/subscriptions`	Create subscription
GET	`/subscriptions/current`	Get current subscription
POST	`/subscriptions/change-plan`	Request plan change
POST	`/subscriptions/cancel`	Cancel subscription
POST	`/webhooks/payments/:provider`	Process authenticated gateway webhook

Platform administration endpoints MUST use separate platform-level permissions. Tenant users MUST NOT be able to invoke platform-wide operations.

11.11 API implementation rules

- Use cursor or bounded page-based pagination.
- Validate request schemas with a shared schema library.
- Enforce tenant scope within repository/service queries.
- Use idempotency keys for order creation where retries may occur, payments and refund initiation.
- Use optimistic concurrency controls for sensitive updates.
- Return HTTP status codes consistently.
- Version breaking API changes.
- Document endpoints using OpenAPI.
- Protect webhook routes using provider signature verification and replay protection.
- Do not rely on a frontend-supplied organization ID as proof of authorization.

12. Role and Permission Matrix

Permission checks MUST be implemented on the backend.

Legend:

- **Full:** All authorized actions in the assigned scope.
- **Limited:** Actions restricted by role, assignment, branch or workflow.
- **Approve:** Approval-only access.
- **Own:** Customer's own permitted records.
- **No:** No access by default.

Capability	Platform admin	Owner	Branch manager	Reception	Operator	Inspector	Cashier	Driver	Accountant
Manage SaaS tenants	Full	No	No	No	No	No	No	No	No
Organization settings	Limited	Full	Limited	No	No	No	No	No	No
Branch configuration	Platform scope	Full	Limited	No	No	No	No	No	No
Customer records	Audited support	Full	Limited	Limited	Limited	Limited	Limited	Assigned only	Limited
Create orders	No by default	Full	Full	Full	No	No	Limited	No	No
Edit prices	No by default	Full	Limited	No	No	No	No	No	No
Process garments	No by default	Full	Full	Limited	Full	Limited	No	No	No
Approve quality control	No by default	Full	Limited	No	No	Full	No	No	No
Collect payments	No by default	Full	Limited	Limited	No	No	Full	Limited	Limited
Approve refunds	No by default	Full	Limited	No	No	No	Limited	No	Limited
Inventory operations	No by default	Full	Full	No	Limited	No	No	No	Limited

Delivery operations	No by default	Full	Full	Limited	No	No	No	Assigned only	No
View financial reports	Audited support	Full	Limited	No	No	No	Limited	No	Full
Manage staff	Platform scope	Full	Limited	No	No	No	No	No	No
View audit logs	Platform scope	Full	Limited	No	No	No	Limited	No	Limited

This matrix is a starting baseline. Final permissions MUST be defined at action level, including create, read, update, cancel, approve, export and administer.

13. Security and Privacy Requirements

13.1 Authentication and authorization

- Use secure password hashing.
- Use secure, HTTP-only session cookies where cookie-based authentication is chosen.
- Apply appropriate `Secure` and `SameSite` cookie attributes.
- Protect cookie-authenticated state-changing requests against CSRF.
- Enforce authorization on every protected backend operation.
- Support session revocation and account suspension.
- Apply least privilege and branch-level restrictions.
- Require stronger authentication for high-risk administration.

13.2 Tenant isolation

- Validate tenant ownership for every resource.
- Scope database queries to authorized organizations.
- Verify parent-child ownership for nested resources.
- Protect file downloads and exports with the same tenant rules.
- Include tenant scope in background jobs and event payloads.
- Test cross-tenant access using adversarial test cases.

- Restrict support impersonation and record every use.

13.3 Application security

- Validate all input.
- Use parameterized database queries.
- Apply rate limiting.
- Protect against common web application vulnerabilities.
- Apply a suitable Content Security Policy.
- Keep dependencies patched.
- Scan dependencies and container images.
- Store secrets outside source code.
- Prevent sensitive data from appearing in logs.
- Review file upload type, size and content.
- Use short-lived, authorized access to private file objects.

13.4 Financial security

- Never trust browser redirects as payment confirmation.
- Verify webhook signatures.
- Enforce idempotency and transaction uniqueness.
- Preserve original payment records.
- Record refund approvals.
- Restrict manual adjustments.
- Reconcile payment provider transactions.
- Avoid storing card data unless a separately approved, compliant architecture requires it.

13.5 Privacy

The system SHOULD collect only data needed for service delivery, operations and lawful business requirements.

It MUST support appropriate notice, access control, data retention, consent handling and secure deletion or anonymization where legally permissible.

The product owner MUST validate applicable Indian privacy and data-protection obligations, including the Digital Personal Data Protection Act and applicable rules, with qualified counsel before production.

14. Non-Functional Requirements

14.1 Performance

Initial targets under the agreed pilot workload:

- Common read APIs: p95 response time below 500 ms.
- Common write APIs: p95 response time below 800 ms, excluding external provider latency.
- Barcode lookup: p95 response time below 300 ms under the target workload.
- Long-running exports and large reports SHOULD execute asynchronously.

These are targets to validate through load testing, not existing performance guarantees.

14.2 Availability and reliability

- Initial availability target: 99.5% monthly, with defined exclusions.
- Critical transactions MUST be atomic.
- Background jobs MUST support controlled retries.
- The system MUST detect failed integration jobs.
- Failed notifications MUST NOT invalidate successful business transactions.

14.3 Scalability

The architecture MUST support growth in:

- Organizations.
- Branches.
- Users.
- Customers.
- Orders.
- Garment history.
- Payment records.
- Notification volume.
- Report workloads.

Performance optimization SHOULD be based on measured workloads and query plans.

14.4 Accessibility and usability

- Responsive desktop, tablet and mobile layouts.
- Keyboard accessibility for essential workflows.
- Accessible labels and validation messages.
- Appropriate contrast.
- Clear focus indicators.

- Screen-reader-friendly form controls.
- Consistent navigation and error handling.

The product SHOULD target WCAG 2.2 AA where practical.

14.5 Localization

The initial market configuration SHOULD include:

- English.
- INR.
- Asia/Kolkata timezone as a configurable default.
- Indian date and number formatting.
- Configurable invoice language and business details.
- Extensible translation support.

Currency, timezone and locale MUST NOT be hard-coded as universal constants.

14.6 Recovery targets

Proposed initial objectives:

- Recovery Point Objective (RPO): no more than 15 minutes of data loss.
- Recovery Time Objective (RTO): service restoration within 4 hours.

These targets require an appropriate backup and replication design, operational staffing and regular restore drills. They MUST be confirmed against budget and infrastructure before approval.

15. Testing Strategy

15.1 Unit testing

Test:

- Price calculations.
- Discount and tax calculations.
- Payment allocation.
- Refund limits.
- Balance calculations.
- State transitions.
- Role and permission checks.

- Inventory movement calculations.
- Subscription entitlements.

15.2 Integration testing

Test:

- PostgreSQL transaction behavior.
- Drizzle migrations.
- Redis queues.
- Payment provider integration.
- Webhook verification.
- Notification provider integration.
- Object storage access.
- Printer and label output.
- Authentication and session revocation.

15.3 End-to-end testing

The test suite MUST cover:

1. Customer registration and lookup.
2. Order creation with multiple garments.
3. Label printing and barcode scanning.
4. Processing through all configured stages.
5. Quality rejection and rework.
6. Partial completion and delivery.
7. Advance and split payments.
8. Refund approval and reconciliation.
9. Notification failure and retry.
10. Branch transfer.
11. Tenant subscription and suspension.
12. Cross-tenant access rejection.

15.4 Security testing

Test:

- Authentication bypass attempts.
- Broken object-level authorization.
- Cross-tenant record access.
- Privilege escalation.
- CSRF and XSS protections.

- Rate limits.
- Malicious uploads.
- Webhook replay.
- Duplicate payment requests.
- Export authorization.
- Session revocation.
- Secrets and sensitive data exposure.

15.5 Performance testing

Simulate:

- Concurrent counter users.
- Large customer directories.
- High-volume barcode scans.
- Simultaneous order creation.
- Payment webhook bursts.
- Large reporting queries.
- Background notification backlog.

The product owner and engineering team MUST approve target user counts, order volume and dataset size before performance acceptance.

16. Acceptance Criteria

A release MUST satisfy the following criteria before production approval.

AC-01 — Tenant isolation

Given two different organizations, a user from one organization cannot retrieve, modify, export or delete the other organization's records through any API or file-access path.

AC-02 — Customer management

An authorized employee can create a customer, find the profile, open its order history and create a new order without duplicating the customer unnecessarily.

AC-03 — Order creation

A valid order is created with its garment records, price snapshots and required initial workflow records consistently. A failed operation does not leave an inconsistent partial transaction.

AC-04 — Garment tracking

Each garment has a unique tag. Valid transitions are recorded with actor and timestamp, and invalid transitions are rejected.

AC-05 — Pricing

The same order input and pricing configuration produce consistent totals. Future price changes do not alter historical invoices.

AC-06 — Payment integrity

Partial payments, repeated requests, refunds and gateway callbacks do not create duplicate financial transactions. Invoice balances reconcile with valid charges and allocations.

AC-07 — Quality control

Garments requiring inspection cannot pass through the normal release workflow without the required approval.

AC-08 — Delivery

The system records dispatch and delivery confirmation. Partial delivery does not incorrectly close the remaining garments.

AC-09 — Notifications

Notification failures are recorded and retried according to policy. A messaging failure does not undo an otherwise successful order.

AC-10 — Reporting

Financial reports reconcile with the underlying transactions using documented definitions.

AC-11 — Permissions

Unauthorized operations are rejected by the backend even when requests are made directly without using the frontend.

AC-12 — Recovery

A backup restoration exercise demonstrates that critical data can be restored within the approved recovery objectives.

AC-13 — Operational readiness

Monitoring, alerting, support procedures, backup verification and rollback procedures are documented and tested.

17. Deployment and DevOps Specification

17.1 Environments

The system MUST maintain separate:

- Local development.
- Automated test/CI.
- Staging.
- Production.

Production data MUST NOT be copied into development environments without an approved, appropriately protected process.

17.2 Deployment architecture

Initial production deployment SHOULD include:

- TLS-enabled reverse proxy.
- Next.js web application.
- Fastify API service.
- PostgreSQL database with protected access.
- Redis for queues and caching.
- Background worker process.
- Private object storage.
- Centralized application logs.
- Monitoring and error alerts.
- Automated backup storage.

Services SHOULD be containerized and configured using environment-specific settings.

17.3 CI/CD pipeline

Every production-bound change SHOULD pass:

1. Formatting and lint checks.
2. Type checking.
3. Unit tests.
4. Integration tests.
5. Dependency and security scans.
6. Production build.
7. Migration validation.
8. Staging deployment.
9. Smoke tests.
10. Approved production release.

17.4 Database migrations

- Use version-controlled Drizzle migrations.
- Review destructive migrations.
- Back up before high-risk schema changes.
- Prefer backward-compatible expand-and-contract migrations.
- Test migrations on staging data of representative size.
- Do not automatically run unreviewed schema pushes in production.
- Document recovery procedures for failed migrations.

17.5 Backups

- Perform scheduled PostgreSQL backups.
- Store backups separately from the primary database.
- Protect backup credentials.
- Encrypt backups in transit and at rest.
- Define retention periods.
- Monitor backup failures.
- Perform regular restore drills.
- Document point-in-time recovery where supported.

17.6 Monitoring and alerting

Monitor:

- API error rate and latency.
- Database health and connection saturation.

- Redis availability and queue backlog.
- Background job failures.
- Payment webhook failures.
- Notification delivery failures.
- Disk and memory utilization.
- Backup success.
- Authentication anomalies.
- Tenant isolation or authorization failures.

17.7 Production release process

1. Approve release scope.
2. Confirm staging test results.
3. Verify backups.
4. Apply compatible migrations.
5. Deploy application services.
6. Run smoke tests.
7. Monitor critical workflows.
8. Roll back application changes if required.
9. Execute documented data recovery procedures when necessary.
10. Record release and incident information.

18. Implementation Milestones

The following estimates assume a small experienced team and phased delivery. They are planning estimates and must be revised after discovery.

Phase 0 — Discovery and product definition

Estimated duration: 1 week

Deliverables:

- Approved business workflows.
- Stakeholder interviews.
- User stories.
- Scope and exclusions.
- Initial acceptance criteria.
- Architecture decision record.
- Prioritized backlog.

Exit criteria:

- Product owner approves the baseline scope.

Phase 1 — Foundation and tenant architecture

Estimated duration: 1–2 weeks

Deliverables:

- Repository and application structure.
- Authentication and sessions.
- Organizations and branches.
- Roles and permissions.
- Tenant-scoped database design.
- Audit framework.
- CI/CD and development environments.

Exit criteria:

- Cross-tenant isolation tests pass.
- Initial deployment pipeline works.

Phase 2 — Customers, catalogue and POS

Estimated duration: 2 weeks

Deliverables:

- Customer management.
- Addresses and customer history.
- Garment and service catalogue.
- Pricing and tax configuration.
- Order creation.
- Invoice and receipt generation.
- Initial payment records.

Exit criteria:

- Customer-to-order workflow passes end-to-end testing.

Phase 3 — Garment operations

Estimated duration: 1–2 weeks

Deliverables:

- Garment-level records.
- Barcode and QR labels.
- Production queues.
- Processing transitions.
- Quality inspection.
- Rework.
- Packing and storage.

Exit criteria:

- Garments remain traceable through all configured stages.

Phase 4 — Payments and fulfillment

Estimated duration: 1–2 weeks

Deliverables:

- Payment gateway integration.
- Partial payment and refund workflows.
- Reconciliation.
- Pickup and delivery scheduling.
- Driver worklist.
- Notification integration.

Exit criteria:

- Payment, collection and delivery workflows reconcile correctly.

Phase 5 — Business management and reporting

Estimated duration: 1–2 weeks

Deliverables:

- Inventory.
- Supplier and purchasing records.
- Expenses.
- Cashier shift closing.
- Operational reports.
- Financial dashboards.
- Complaints and claims.

Exit criteria:

- Core reports reconcile with source transactions.

Phase 6 — SaaS commercialization

Estimated duration: 1–2 weeks, potentially overlapping earlier phases

Deliverables:

- Subscription plans.
- Entitlements.
- Subscription lifecycle.
- Platform administration.
- Tenant provisioning.
- Usage limits.
- Suspension and reactivation.

Exit criteria:

- Subscription state and entitlements behave correctly under normal and failure conditions.

Phase 7 — Hardening and pilot

Estimated duration: 1–2 weeks

Deliverables:

- Security testing.
- Performance testing.
- Restore exercise.
- Hardware compatibility testing.
- User acceptance testing.
- Staff training materials.
- Operational documentation.
- Production release approval.

Exit criteria:

- All release-blocking defects are resolved or formally accepted with documented mitigation.

Overall planning estimate: Approximately 10–14 weeks with overlapping work and a small experienced team. More complex integrations, hardware constraints, regulatory requirements or a larger initial scope may extend delivery.

19. Project Deliverables

The development team MUST produce:

1. Approved SRS.
2. UX wireframes and design system.
3. Architecture diagrams and decision records.
4. PostgreSQL schema and Drizzle migrations.
5. API specification and OpenAPI documentation.
6. Frontend application.
7. Backend application.
8. Background worker implementation.
9. Automated test suite.
10. Role-permission matrix.
11. Deployment and environment configuration.
12. CI/CD pipeline.
13. Monitoring and alerting configuration.
14. Backup and restore procedures.
15. User and administrator documentation.
16. QA report and acceptance evidence.
17. Release checklist and rollback plan.
18. Data migration procedures.
19. Support and incident-response procedures.
20. SaaS onboarding and subscription administration documentation.

20. Change Management

Requirements may change as the business is validated.

Each change request MUST document:

- Request identifier.
- Business justification.
- Affected requirements.
- UI and API impact.
- Database migration impact.
- Security and tenant-isolation impact.
- Testing impact.
- Estimated effort.
- Schedule and cost impact.
- Approval decision.

Changes affecting financial records, tenant isolation, permissions, data retention or payment processing MUST receive additional engineering and security review.

21. Definition of Done

A feature is complete only when:

- The requirement is implemented.
- Backend validation and authorization are in place.
- The UI supports loading, success, empty and error states.
- Database migrations are reviewed.
- Unit and relevant integration tests pass.
- API documentation is updated.
- Audit requirements are implemented.
- Tenant isolation has been verified where applicable.
- Logging and operational monitoring are sufficient.
- QA acceptance criteria pass.
- The feature is deployed to staging and reviewed.
- Required documentation is updated.

A frontend-only implementation MUST NOT be considered complete when the feature requires backend persistence, authorization or financial integrity.

22. Final Product Acceptance

The product owner may approve the first commercial release when:

1. All P0 requirements are complete.
2. Core order, garment and payment workflows pass acceptance tests.
3. Tenant and branch authorization tests pass.
4. Financial reconciliation has been verified.
5. Critical security issues are resolved.
6. Production monitoring and backup procedures are operational.
7. The agreed recovery objectives have been tested.
8. Staff training and user documentation are available.
9. Outstanding defects are documented and accepted according to severity.
10. A controlled rollout and rollback plan is approved.

23. Recommended Initial Build Order

The implementation team SHOULD execute development in the following order:

1. Project foundation and tenant isolation.
2. Authentication, users, roles and branches.
3. Customer management.
4. Garment catalogue, services and pricing.
5. Order intake and garment-level records.
6. Barcode labels and production workflow.
7. Quality control, rework, packing and storage.
8. Invoices, payments, balances and refunds.
9. Pickup, delivery and notifications.
10. Inventory, suppliers and expenses.
11. Reporting and dashboards.
12. SaaS subscription management.
13. Platform administration.
14. Security hardening, load testing and production rollout.

This sequence prioritizes a working operational system while preserving the architecture required for multi-tenant commercial growth.

End of SRS — Version 1.0

Approval status: Proposed baseline. Product owner and engineering lead approval is required before the specification is treated as the final contractual scope.

Next engineering artifact: A detailed implementation backlog containing epics, user stories, screen-level tasks, database migrations, API contracts, test cases and release dependencies derived from this SRS.